

PRÁCTICAS 1 & 3

Reproductor Multimedia y comunicación WebRTC en tiempo real

Gestió i Distribució de la Informació Empresarial (GDIE)

Práctica 1	Reproductor multimedia HTML5, MSE, VTT, IA offline
Práctica 3	WebRTC con señalización WebSocket, Quiz interactivo P2P
Tecnologías	HTML · CSS · JavaScript · Node.js · Express · Socket.IO · WebRTC
Servidor	https://gdie2603.ltim.uib.es
Autores	Rafel Mulet, Juana Maria Luna
Fecha	08 de Junio del 2026

PRÁCTICA 1 — REPRODUCTOR MULTIMEDIA INTERACTIVO

INTRODUCCIÓN

La práctica consiste en desarrollar un reproductor multimedia interactivo en una página web utilizando HTML, CSS y JavaScript. A diferencia de un reproductor normal, se ha creado una interfaz propia, con controles personalizados (play, pausa, volumen, progreso, calidad, subtítulos) en lugar de los controles nativos del navegador. Además, se integran archivos VTT, que permiten añadir no solo subtítulos, sino también información semántica sincronizada con el vídeo, como datos adicionales y eventos interactivos (quiz). Durante la reproducción, el sistema puede mostrar contenido dinámico, como información contextual o preguntas tipo quiz, haciendo que el vídeo sea interactivo.

ELABORACIÓN

Vídeo

El contenido del vídeo se ha extraído de un vídeo de YouTube en el que se muestran los distintos deportes en las olimpiadas de París 2024. Se han cortado fragmentos de 18 segundos de cada vídeo con keyframes cada 2 segundos (y posteriormente fragmentos de 6 en MPEG-DASH y HLS). Enlace al vídeo y comando de ffmpeg usado a continuación:

<https://youtu.be/V1g3fdV1hvk?si=EmILGEIBhW9qTJjN>

```
ffmpeg -i olimpiades.mp4 -ss 00:06:04 -to 00:06:22 -c:v libx264 -c:a aac  
olimpiades_frag9.mp4
```

Posteriormente se han igualado los metadatos necesarios para unir los vídeos mediante el siguiente comando:

```
ffmpeg -i olimpiades_frag1.mp4 -c:v libx264 -crf 18 -preset slow -r 30 -vf  
"scale=3840:2160,format=yuv420p" -video_track_timescale 90000 -c:a aac -ar 48000  
-ac 2 norm_frag1.mp4
```

Finalmente, se han unido los fragmentos para tener un solo vídeo con el siguiente comando y se han extraído 5 vídeos (sin audio) y 3 audios de distintas calidades:

```
ffmpeg -f concat -safe 0 -i videos.txt -c copy OlympicsSportsV1_4K.mp4
```

```
ffmpeg -i OlympicsSportsV1_4K.mp4 \  
-filter_complex "[0:v]split=5[v2k][vfh][vhd][v480][v360]; \  
[v2k]scale=w=2560:h=1440[v2kout]; \  
[vfh]scale=w=1920:h=1080[vfhout]; \  
[vhd]scale=w=1280:h=720[vhdout]; \  
[v480]scale=w=854:h=480[v480out]; \  
[v360]scale=w=640:h=360[v360out]" \  
-ac 0 -c:v v2kout,vfhout,vhdout,v480out,v360out -c:a aac -ar 48000 -ac 3
```

```
-map "[v2kout]" -c:v libx264 -crf 23 -b:v 10M -maxrate 10M -bufsize 20M -
preset slow -g 60 -sc_threshold 0 -keyint_min 60 video_2k.mp4 `
-map "[vfhout]" -c:v libx264 -crf 23 -b:v 7M -maxrate 7M -bufsize 14M -
preset slow -g 60 -sc_threshold 0 -keyint_min 60 video_1080p.mp4 `
-map "[vhdout]" -c:v libx264 -crf 23 -b:v 3M -maxrate 3M -bufsize 6M -
preset slow -g 60 -sc_threshold 0 -keyint_min 60 video_720p.mp4 `
-map "[v480out]" -c:v libx264 -crf 23 -b:v 1.5M -maxrate 1.5M -bufsize 3M -
preset slow -g 60 -sc_threshold 0 -keyint_min 60 video_480p.mp4 `
-map "[v360out]" -c:v libx264 -crf 23 -b:v 1M -maxrate 1M -bufsize 2M -
preset slow -g 60 -sc_threshold 0 -keyint_min 60 video_360p.mp4 `
-map a:0 -c:a aac -b:a 320k -ac 2 audio_high.aac `
-map a:0 -c:a aac -b:a 128k -ac 2 audio_normal.aac `
-map a:0 -c:a aac -b:a 64k -ac 2 audio_low.aac
```

Página web

Se ha elaborado una página web que muestra el vídeo y la distinta información de los deportes. El vídeo consta de una barra de control con la que se pueden realizar distintas funciones.

- STOP: volver al inicio con la configuración predeterminada de la barra de controles.
- PLAY/PAUSE: botón que inicia y/o pausa el vídeo.
- CONTROL DE VOLUMEN: botón que permite mutear el vídeo. Cuando el ratón se coloca encima se muestra un pequeño deslizable que permite ajustar el nivel de volumen.
- BARRA DE TIEMPO: barra que va avanzando mientras pasa el tiempo del vídeo. Se permite desplazarse a cualquier momento del vídeo con esta barra.
- CALIDAD DE AUDIO: opciones de distintas calidades de audio.
 - HI: cambia la calidad de audio a alta.
 - MD: cambia la calidad de audio a media.
 - LO: cambia la calidad de audio a baja.
- CALIDAD DE VÍDEO:
 - 2K: cambia la calidad de vídeo a 2K.
 - 1080P: cambia la calidad de vídeo a 1080p.
 - 720p: cambia la calidad de vídeo a 720p.
 - 480p: cambia la calidad de vídeo a 480p.
 - 360p: cambia la calidad de vídeo a 360p.
- SUBTÍTULOS: permite activar o desactivar los subtítulos. Están disponibles en inglés o en español.

Durante el transcurso del vídeo se muestran distintas preguntas de los deportes y fragmentos que se van mostrando. Estas preguntas constan de 4 opciones, con una sola respuesta correcta.

La página web consta de 4 ficheros de metadatos:

- Subtítulos en inglés.
- Subtítulos en español.
- Capítulos de los vídeos con metadatos.
- Preguntas del quiz.

Ficheros VTT y semántica

Se han creado cuatro ficheros VTT que cubren todos los requisitos técnicos del enunciado:

Fichero	kind	Descripción
subs_en.vtt	subtitles	Subtítulos en inglés generados con speech-to-text.cloud
subs_es.vtt	subtitles	Subtítulos en español
metadata.vtt	metadata	Metadatos semánticos JSON sincronizados: título del deporte, descripción, países origen y top 3
quiz.vtt	metadata	Preguntas del quiz: texto, 4 opciones y índice correcto

Cada cue del fichero metadata.vtt contiene un objeto JSON con la siguiente estructura:

```
{
  "title": "Atletismo",
  "desc": "Pruebas de velocidad, salto y lanzamiento.",
  "p_mas": { "f": "img/usa.png", "n": "Estados Unidos" },
  "p_ori": { "f": "img/gre.png", "n": "Grecia" },
  "tops": [
    { "p": "img/gold.png", "n": "Marcell Jacobs" },
    { "p": "img/silver.png", "n": "Fred Kerley" },
    { "p": "img/bronze.png", "n": "Andre De Grasse" }
  ]
}
```

Los cues del fichero quiz.vtt siguen un esquema con los campos question, options (array de 4 strings) y correctIndex para que el motor semántico pueda validar la respuesta:

```
{
  "question": "¿Cuántas pruebas de atletismo hay en los JJOO?",
  "options": ["24", "36", "48", "54"],
  "correctIndex": 1
}
```

USO DE IA OFFLINE

Se ha usado inteligencia artificial en dos ocasiones:

1. Para obtener el fichero VTT con los subtítulos en inglés se ha usado una página web que convierte audio a texto y permite exportarlo directamente a formato WEBVTT:

<https://www.speech-to-text.cloud/>

2. Se ha usado Gemini para obtener el VTT de las preguntas del quiz. Se le proporcionó el vídeo y el formato exacto que debía tener cada pregunta en el fichero.

MPEG-DASH Y HLS

La implementación del cambio de calidad de vídeo y audio ha sido la parte más compleja de la práctica. Se han obtenido los ficheros chunk, init y manifest de las distintas calidades, todos en fragmentos de 6 segundos, mediante el siguiente comando:

```
ffmpeg -y `
-i video_360p.mp4 -i video_480p.mp4 -i video_720p.mp4 -i video_1080p.mp4 -i
video_2k.mp4 `
-i audio_low.aac -i audio_normal.aac -i audio_high.aac `
-map 0:v -map 1:v -map 2:v -map 3:v -map 4:v `
-map 5:a -map 6:a -map 7:a `
-c copy `
-f dash `
-seg_duration 6 `
-use_template 1 -use_timeline 1 `
-dash_segment_type mp4 `
-adaptation_sets "id=0,streams=v id=1,streams=a" `
-init_seg_name 'init-$RepresentationID$.m4s' `
-media_seg_name 'chunk-$RepresentationID$-$Number%05d$.m4s' `
out/manifest.mpd
```

Posteriormente, se ha modificado el código para que al seleccionar el botón de las flechas (DASH) en las opciones de video o audio se active esta implementación. Se ha conseguido que funcione, pero no con la fluidez esperada.

PRUEBAS

Caso de prueba	Resultado esperado	Resultado obtenido
loadedmetadata se dispara al cargar	Duración y dimensiones visibles en consola	Correcto en todos los navegadores probados
Play / Pause sin desincronizar UI	Icono cambia correctamente, audio y vídeo sincronizados	Correcto
Seek adelante y atrás	El vídeo retoma la posición correcta y el audio se sincroniza	Correcto. Drift < 0.15 s gestionado por syncMedia()
Subtítulos EN / ES activables	Pista correcta visible sobre el vídeo	Correcto
metadata genera eventos visibles (show_overlay)	Panel lateral actualiza título, descripción y podio	Correcto. 9 deportes con datos distintos
Quiz pausa vídeo y muestra pregunta	Vídeo se detiene, controles bloqueados, pregunta visible	Correcto

PRÁCTICA 3 — WEBRTC CON SERVIDOR DE SEÑALIZACIÓN

INTRODUCCIÓN

Esta práctica es una continuación de la Práctica 1, en la que se trabajó la base de una aplicación web cliente-servidor. En esta nueva práctica se amplía esa arquitectura incorporando comunicación multimedia en tiempo real entre navegadores mediante WebRTC, una tecnología que permite transmitir datos directamente entre clientes sin pasar por el servidor.

La idea principal es comprender la diferencia entre comunicación cliente-servidor y comunicación peer-to-peer, el proceso de señalización necesario para iniciar una conexión WebRTC, y el uso de RTCDataChannel para enviar mensajes de control entre los dos navegadores.

Nuestra práctica conecta dos páginas (el Visualizador y el Respondedor) y permite que cuando el vídeo llega a un marcador de quiz, la pregunta se envíe directamente al dispositivo Respondedor vía RTCDataChannel. Una vez el usuario responde, el Visualizador reanuda automáticamente el vídeo.

ELABORACIÓN

Arquitectura del sistema

La práctica introduce tres capas de comunicación sobre la base de la Práctica 1:

Tecnología	Tipo de comunicación	Función
Express (HTTP)	Cliente ↔ Servidor	Servicio de todos los ficheros estáticos (HTML, CSS, JS, media, tracks)
Socket.IO / WebSocket	Cliente ↔ Servidor	Señalización: join, offer, answer, ICE candidates
WebRTC (RTCPeerConnection)	Peer-to-peer	Canal de datos directo entre Visualizador y Respondedor
RTCDataChannel	Peer-to-peer	Mensajes SHOW_QUIZ y RESUME_VIDEO sin pasar por el servidor

El servidor de señalización no transmite en ningún momento el audio ni el vídeo. Su único papel es coordinar el intercambio inicial de información SDP e ICE para que los navegadores puedan comunicarse directamente.

Servidor de señalización (server.js)

El servidor está implementado con Node.js, Express y Socket.IO. Mantiene un objeto `rooms` en memoria que mapea cada sala con los socket IDs de los dos clientes:

```
const rooms = {}; // rooms['sala-quiz'] = ['socketId_A', 'socketId_B']
```

Gestión de eventos:

- `join`: un cliente solicita unirse a la sala. Si ya hay 2 clientes se emite `full`. Cuando se completa la sala se emite `ready` al primero (Visualizador) para que inicie el offer.
- `message`: reenvía al otro socket de la sala el mensaje recibido (offer, answer o candidato ICE) sin interpretar su contenido.
- `disconnect`: elimina al cliente de la sala y emite `peer-disconnected` al cliente restante.

```
socket.on('message', ({ room: roomId, message }) => {  
  socket.to(roomId).emit('message', message);  
});
```

RTCDataChannel: mensajes de quiz

Una vez establecida la conexión P2P, toda la comunicación de datos entre los dos navegadores se realiza a través del canal de datos, sin pasar por el servidor. Se intercambian dos tipos de mensajes:

Mensaje	Dirección	Descripción
SHOW_QUIZ	Visualizador → Respondedor	JSON con la pregunta, las 4 opciones y el índice correcto
RESUME_VIDEO	Respondedor → Visualizador	Indica que el usuario ha respondido y el vídeo puede reanudarse

El Visualizador crea el `DataChannel` antes de generar el offer:

```
dataChannel = peerConnection.createDataChannel('quizChannel');
```

El Respondedor lo recibe mediante el evento `ondatachannel`:

```
peerConnection.ondatachannel = (event) => { dataChannel = event.channel; };
```

Si el vídeo llega a un marcador de quiz antes de que el canal esté abierto, el quiz se guarda en la variable `pendingQuiz` y se envía en cuanto el canal reporta el evento `onopen`:

```
dataChannel.onopen = () => {  
  if (pendingQuiz) {  
    dataChannel.send(JSON.stringify({ type: 'SHOW_QUIZ', payload:  
pendingQuiz }));  
    pendingQuiz = null;  
  }  
};
```

Funcionamiento del quiz interactivo

El flujo completo de una pregunta de quiz es el siguiente:

- El vídeo avanza y el cue del quiz se activa en la pista VTT de metadatos (label='Quiz').
- El Visualizador pausa el vídeo, bloquea los controles y envía el JSON de la pregunta al Respondedor vía RTCDataChannel.
- El Respondedor muestra la pregunta con cuatro botones de opción.
- El usuario selecciona una respuesta: se muestra feedback en verde (correcto) o rojo (incorrecto) y se marca la opción correcta.
- Tras 2,5 segundos, el Respondedor envía RESUME_VIDEO al Visualizador.
- El Visualizador desbloquea los controles y reanuda el vídeo.

CONCLUSIONES

En conclusión, estas prácticas nos han permitido comprender mejor cómo funciona un sistema multimedia moderno, desde la preparación y reproducción de contenidos hasta la interacción entre usuarios en tiempo real.

En la primera práctica hemos desarrollado un reproductor multimedia con HTML5, incorporando funcionalidades como capítulos, metadatos sincronizados y cuestionarios interactivos mediante archivos VTT. Gracias a ello, hemos visto cómo la información asociada al vídeo puede mejorar la experiencia de usuario y aportar nuevas formas de interacción.

En la tercera práctica hemos añadido comunicación en tiempo real mediante WebRTC, permitiendo compartir cuestionarios y controlar la reproducción directamente entre navegadores con una latencia muy baja.

En conjunto, ambas prácticas se han complementado muy bien y como resultado, hemos obtenido una aplicación más completa e interactiva que combina reproducción multimedia, metadatos sincronizados y comunicación en tiempo real.